

```
pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

```
def predict_odd_even(hours_studied):
    return [1 if h[0] % 2 != 0 else 0 for h in hours_studied]
```

```
data={
    'hours_studied':[1,2,3,4,5,6,7,8,9,10],
    'pass_fail':[0,0,0,1,1,1,1,1,1,1]
}
df=pd.DataFrame(data)
x=df[['hours_studied']]
y=df['pass_fail']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print(f'Train:{len(x_train)}rows | Test:{len(x_test)} rows')
#create and train
model=LogisticRegression()
model.fit(x_train,y_train)
print("model trained successfully")
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print(f'Accuracy:{accuracy*100:.2f}%')
print(classification_report(y_test,y_pred))
```

```
Train:8rows | Test:2 rows
model trained successfully
Accuracy:100.00%
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 1       |
| 1            | 1.00      | 1.00   | 1.00     | 1       |
| accuracy     |           |        | 1.00     | 2       |
| macro avg    | 1.00      | 1.00   | 1.00     | 2       |
| weighted avg | 1.00      | 1.00   | 1.00     | 2       |

```
new_students=[[25],[1],[80]]
predictions=model.predict(new_students)
labels = {0: 'Fail', 1: 'Pass'}
for hours,prediction in zip(new_students,predictions):
    print(f"Studied {hours[0]}hrs - {labels[prediction]}")
```

```
Studied 25hrs - Pass
Studied 1hrs - Fail
Studied 80hrs - Pass
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
warnings.warn(
```

```
print("\n--- Predictions using custom odd/even rule ---")
rule_based_predictions = predict_odd_even(new_students)
for hours, prediction in zip(new_students, rule_based_predictions):
    print(f"Studied {hours[0]}hrs - {labels[prediction]}")
```

```
--- Predictions using custom odd/even rule ---
Studied 25hrs - Pass
Studied 1hrs - Pass
Studied 80hrs - Fail
```

```
print(df)
```

|   | hours_studied | pass_fail |
|---|---------------|-----------|
| 0 | 1             | 0         |
| 1 | 2             | 0         |
| 2 | 3             | 0         |
| 3 | 4             | 1         |

|   |    |   |
|---|----|---|
| 4 | 5  | 1 |
| 5 | 6  | 1 |
| 6 | 7  | 1 |
| 7 | 8  | 1 |
| 8 | 9  | 1 |
| 9 | 10 | 1 |

```
from sklearn.datasets import load_iris, load_digits
```

```
#steps  
1.import  
2.prepare data  
3.split  
4.train a model  
5.predict
```